

Ju Zi Hein

SOFTWARE DEVELOPER

LA, USA | (856) 283-0796 | heinjuzi@gmail.com

SUMMARY

- Software Developer with 5+ years of experience in developing robust, scalable applications for fintech and e-commerce sectors.
- Demonstrated proficiency in both front-end and back-end technologies, leveraging Java (Core, Spring Framework, Hibernate) and JavaScript (React.js, Angular 12, Node.js, jQuery) to deliver seamless, high-performance web applications.
- Developed dynamic and scalable web applications using PHP, focusing on robust back-end logic, seamless data integration, and secure user authentication with Laravel and PHP-based frameworks.
- Strong experience with AWS services including EC2, S3, and Lambda, for building efficient and cost-effective cloud-based solutions.
- Skilled in database management and optimization, working with MySQL, MongoDB, and implementing ORM technologies like Hibernate to enhance query performance.
- Implemented comprehensive unit and integration tests with JUnit, Mockito, and Selenium to ensure application reliability and meet stringent performance benchmarks.
- Experienced in Agile methodologies, CI/CD practices, and DevOps tools such as Jenkins, Docker, and Kubernetes for streamlined development and deployment processes.

SKILLS

Programming Languages:	Core Java, C, C++, SQL, C#, GoLang, PHP, Javascript
Frameworks:	Struts, Hibernate, Spring Boot, Spring Batch, Spring Security, Spring AOP, Spring Core, Spring IOC, Angular 12, jQuery, Node.js, React.js, Express.js, Laravel
Web Technologies:	HTML, HTML5, CSS/CSS3, AJAX, jQuery, Bootstrap, XML, JSON, UI Material, SASS, Typescript, ES6, Redux, React Hooks, REST API, RESTful API
Java/J2EE Technologies:	Servlets, Spring, EJB, JPA, JTA, JDBC, JSP, JSTL, Webservices, Microservices, Spring MVC, Hibernate, ORM
Database:	Oracle SQL, MySQL, MongoDB, PostgreSQL, PL/SQL, DynamoDB
Cloud Platforms:	Amazon Web Services
IDEs:	Visual Studio Code, Eclipse IDE, IntelliJ IDEA, Spring Tool Suite (STS)
Web/Application Servers:	Oracle WebLogic, IBM WebSphere, Apache Tomcat, JBoss
Testing Tools:	JUnit, Mockito, Selenium
Build/ Other Tools:	Maven, Gradle, ANT, Jenkins, Docker, Kubernetes, SOAP/REST API, Postman, Jira, SonarQube, Kafka, Jupyter
Methodologies:	SDLC, Agile, Waterfall, SCRUM
Version Control:	Git, GitHub, SVN, Bitbucket
Operating System:	Windows, Linux, Mac OS

EXPERIENCE

Software Developer | PayPal – LA

Aug 2023 – Present

- Worked on the enhancement of the mobile payment processing system, reducing transaction times by 20% and improving overall user experience on mobile platforms and this initiative directly contributed to better customer satisfaction and higher transaction success rates.
- Leveraged Spring Boot's auto-configuration and dependency injection to reduce boilerplate code by 40%, allowing for easy configuration of application components (payment processors, data sources, and messaging queues), which improved maintainability and decreased development time for new features by 30%.
- Designed and implemented responsive, user-friendly web interfaces using ReactJS and JavaScript, improving user flow during payment processing and reducing user friction by 25%, which led to a 20% increase in successful transaction completions and a 15% reduction in bounce rates on mobile and web platforms.
- Utilized MySQL for relational data storage and Hibernate for Object-Relational Mapping. Optimized SQL queries & database access layers to ensure fast and efficient transaction handling, reducing database query execution time by 25%.

- Applied Java Concurrency features like `ExecutorService` and `CompletableFuture` to optimize asynchronous processing of payment transactions, improving system throughput and reducing latency.
- Configured Spring's declarative transaction management to ensure consistency and rollback functionality in the event of failures, thereby maintaining data integrity during high-volume transactions.
- Used Amazon S3 for storing application logs, transaction data, and system artifacts. Implemented an efficient archiving and retrieval system that helped improve debugging and system monitoring for the payment services.
- Employed Apache Kafka as a distributed messaging platform to decouple microservices and implement an event-driven architecture and also, used kafka to manage high-throughput transaction events, allowing seamless communication between front-end, payment processors, and back-end systems.
- Executed Java's multi-threading capabilities to handle concurrent transaction processing, improving system responsiveness during peak usage periods by 35%, allowing for more efficient transaction handling.
- Integrated SLF4J for logging purposes, enabling effective tracking of application behavior and facilitating debugging through structured log messages.
- Actively participated in Agile ceremonies such as sprint planning and daily standups. Collaborated closely with product owners, developers, and QA teams to ensure timely and high-quality deliverables.

Software Developer | Shin Genki Co.Ltd – Yangon

Jun 2018 – Jul 2022

- Managed the development of a web-based inventory management system to automate order processing, real-time stock tracking, and order fulfillment, optimizing the company's supply chain processes.
- Used MongoDB to store non-relational, semi-structured data such as product catalogs, transaction logs, and order histories, enabling flexible schema designs and faster read/write operations.
- Developed a microservices-based architecture for the inventory management system, where each service (product management, order processing, inventory tracking) was decoupled to ensure flexibility and scalability.
- Created two-way data binding with Vue.js to synchronize data between the UI and the backend, ensuring real-time updates on the client side whenever inventory or order details were modified, improving user experience.
- Implemented OAuth 2.0 and JWT (JSON Web Tokens) for securing the APIs, ensuring that only authorized users (admin users or authenticated customers) could access and modify sensitive data such as orders, payment information, and inventory levels.
- Made semantic HTML structures for the inventory management system to enhance accessibility, SEO, and maintainability. Used appropriate HTML tags such as `<header>`, `<footer>`, `<section>`, and `<article>` to ensure clean and well-organized markup.
- Created core business logic for the inventory management system in Java, including functions for managing product stock levels, order processing, and real-time updates, ensuring robust and efficient handling of transactions.
- Designed and implemented pagination and filtering mechanisms for the inventory and order APIs, enabling clients to request large datasets in smaller chunks and filter results based on parameters like product category, order status, or date range, improving API response time by up to 40% and reducing server load by 30%.
- Utilized JSON as the standard format for data exchange between the client and server, ensuring lightweight, human-readable, and easily consumable API responses for inventory and order details.
- Built dynamic, responsive user interfaces using Angular for the inventory management system, improving the user experience by providing real-time updates on product stock, order statuses, and transaction details, resulting in a 25% reduction in user drop-off rates and a 35% increase in user engagement.
- Leveraged Spring Boot's transaction management capabilities to ensure ACID compliance for financial transactions (orders and payments), maintaining data consistency even in the event of system failures or errors, reducing transaction failures by 30% and improving overall system reliability by 20%.
- Created AJAX (Asynchronous JavaScript and XML) to fetch data from back-end APIs asynchronously, reducing page load times and providing a smoother, non-blocking user experience when updating inventory or order details.
- Integrated JUnit, Mockito, and Selenium tests into the CI pipeline to run automated unit tests, integration tests, and end-to-end tests every time new code was pushed, ensuring that no defects were introduced into the system.
- Maintained detailed change logs and used annotations within the code to keep Swagger documentation up to date with API modifications, ensuring accuracy.
- Utilized Mockito to create mock objects for unit testing, allowing for isolation of components and testing individual methods without dependencies on external systems.

EDUCATION

Master of Science in Computer Science – California State University, Los Angeles, USA

Bachelor of Technology in Computer Science – University of Information Technology, Myanmar